TEAM KRATOS-X · PROJECT OVERVIEW

Lefty

An AI assistant for Windows that does it with you, in any app. He shows you where to click, or does it for you, and checks every step.

November 2026 · Version 0.1.5

1. What Lefty is

Lefty lives on your Windows desktop. Press **Ctrl+Space** in any app, type or say what you want to do, and Lefty gets it done with you, step by step, until the task is actually finished.

He is **not a chatbot**. A chatbot tells you the steps in text and you still do all the work. Lefty works inside your real apps: he finds the actual buttons on screen, points at them or clicks them, and checks that each step worked before moving on.

Two modes

Mode	What happens
Learn	Lefty points at the next button. You click it, and he checks it worked before showing the next step. You learn the app by doing it.
Do it	Lefty clicks and types for you while you watch. He asks before anything risky (delete, buy, send, install), never types into password boxes, and stops the moment you press Esc, click or type.

You can switch between the modes in the middle of a task: "Do it for me" from any step, or "Guide me instead" after stopping him.

2. How Lefty works

1. Read Windows UI Automation gives the real list of buttons on screen, with names and positions. →

2. Decide The AI brain picks the one next step toward your goal. →

3. Act He points at it (Learn) or clicks and types it (Do it). →

4. Check He confirms the screen changed as expected, then repeats.

Because Lefty reads real, named buttons instead of guessing from pixels, he works in almost any Windows app with no setup for each app. A screenshot is only sent to the AI when a step needs it, and Lefty hides

himself from it.

3. What makes Lefty different

> **Other assistants do it for you. Lefty teaches you to do it yourself, in any app, even offline.**

- **Teaching first.** Learn mode guides you and checks every step. Most AI agents only take over.
- **Type-along code.** Code appears as faint "ghost" text and you type it yourself, while Lefty tracks every character. Copilot writes code for you; Lefty teaches you to write it.
- **Learn and do in one.** Switching between being taught and having it done, mid-task, is rare.
- **Any Windows app, no setup.** Company training tools need setup for each app; Lefty reads apps through Windows itself.
- **Our own brain, offline** IN PROGRESS Lefty will run on open AI models on our own laptops, so he works with Wi-Fi off and your screen never leaves your machines.

4. What works today

Feature	What it does
Guided steps	Step-by-step help in any app, even with typos in the request.
Do it mode	Clicks, types, fills Save and Open dialogs, asks before risky steps, stops when you take over.
Code help (VS Code)	Type-along ghost code, and "find the error": he rings the wrong line, shows the fix, and offers Fix it for me, I'll type it, or Skip.
Website builder	"Build me a portfolio website" writes every file into a folder you choose and opens it in VS Code or the browser. Replaced files are backed up first.
Voice	Hold Ctrl+Space and say your goal, in any language. He sends it when you stop talking.
Focus mode	"Focus 30 minutes on VS Code and physics" closes anything else that comes to the front until the time is up.
Chat history	Saved on the PC, or in our own Supabase database. Old chats can be continued.
Installer	One-click Windows installer. Uninstalling closes Lefty and every helper process cleanly.

5. How the AI brain (LLM) works

Today: Gemini WORKING

For every step, Lefty sends the AI one request and gets one step back. The request contains:

- your goal, and the earlier messages in the chat for context;
- the app in front, and the other open apps;
- the steps already done;
- the list of clickable things on screen, each with a number, name, type and position;
- a screenshot, only when the step needs it.

The AI answers in a fixed JSON format, for example:

```
{ "status": "step", "action": "open_app", "instruction": "Open Calculator", "app": "Calculator" }
```

A real request measured about **1,900 tokens in and 50 tokens out**, and came back in about **5 seconds**. (A token is a piece of a word, about three quarters of a word on average.)

Speed tricks. Lefty asks the fastest Flash model first with minimal "thinking". If no answer arrives within 5 seconds, he asks a second model at the same time, and the first good answer wins. If a model is retired, out of quota, overloaded or too slow, he moves to the next one automatically.

Next: our own brain, on our own laptops IN PROGRESS

We download existing open models and run them ourselves with **Ollama**. We do not train a model from scratch; that would take months and millions of dollars. Each laptop has one job and keeps one model loaded, so nothing has to be swapped in and out.

Job	Hardware	Model
Brain	Laptop with an RTX 3050, 6 GB	8B text model: Qwen3 8B (backup: Llama 3.1 8B). Plans steps, picks buttons, answers.
Eyes	Laptop with an RTX 2050, 4 GB	Small 3B vision model, for screenshots, photos and scanned pages. Also runs the Lefty app.
Ears	Laptop with built-in graphics	Whisper speech-to-text, so voice works offline.
Cloud	Google Gemini	Heavy work (long PDFs, big code, places search, images) and automatic fallback.

All laptops connect over our own phone hotspot, so the whole system works **with no internet at all**.

Routing: which brain answers

Request	Goes to
Normal app steps (the button list is text)	8B text model
Apps with no button list, photos, scanned PDFs	3B vision model
Voice	Whisper
Long PDFs, big code files, places search, images	Gemini
Any laptop switched off or unreachable	Gemini takes over

One rule: **each request goes to one model.** Passing every step through two models in a row would be slower than one.

Key ideas behind the setup

- **Text vs vision models.** A text model reads only words. A vision model can also see images, which costs extra graphics memory and time. Lefty's button list is text, so most steps only need the text model.
- **Graphics memory (VRAM) decides the size.** A model must fit in the graphics card's memory to run fast. 4-bit compression shrinks an 8B model to about 5 GB. If a model is bigger than the card, it spills into normal memory and runs several times slower.
- **Context.** Everything the model reads at once. A 4,000-token context is enough for Lefty's requests and keeps the 8B model inside 6 GB.
- **Settings that make it fit and fast:** 4-bit model, 4,096-token context, flash attention, compressed (q8_0) context cache, Qwen3's slow "thinking" mode turned off, and JSON output forced so answers are always valid.
- **No usage limits.** Our own model is free and unlimited; Gemini's free tier has per-minute and per-day limits that reset.

Model size (4-bit)	Memory needed	Fits 4 GB	Fits 6 GB
3-4B with vision	about 3-3.5 GB	Yes	Yes
7-8B text only	about 5-5.5 GB	No	Yes, with the settings above
7-8B with vision	about 6-7 GB	No	Spills over, slow

Memory and speed figures are estimates until benchmarked on our machines.

Making him better: fine-tuning and memory IN PROGRESS

Fine-tuning means taking the downloaded 8B model and teaching it Lefty's specific job with examples of the right answer. It does not need Lefty to be running; examples are produced by a separate script:

1. Open an app and read its real buttons with Lefty's screen reader.
2. Gemini writes many realistic goals for that app.
3. For each goal, Gemini gives the correct step. Goal + buttons + answer = one training example.
4. Repeat across 20-30 apps for 1,000-2,000 examples, and spot-check a sample by hand.

This is called **distillation**: the small model learns from the big one. Training uses **QLoRA**, a low-memory way to fine-tune, on a free Colab or Kaggle GPU, and takes about **1-3 hours**. The result is tested on 20 real tasks that are kept out of the training data, and is used only if it scores higher than the plain model.

Memory gives a faster, instant improvement: a task Lefty has solved once can be replayed directly with no AI call, and he remembers your corrections and preferences.

6. Tools and technologies

Tool	What we use it for
Electron 33 + Node.js	The desktop app: main window, the on-screen overlay and bubble, system tray, screen capture.
Windows UI Automation	Reads every app's buttons, the text cursor and screen state, and performs Do it clicks and key presses. Accessed through a small C# helper, compiled on first run with the C# compiler that ships with Windows.
uiohook-napi	A system-wide keyboard and mouse hook: the Ctrl+Space shortcut, and stopping Lefty when you click or type.
Google Gemini API	The current AI brain: next steps, code, error fixing, understanding speech, reading screenshots.
Node.js backend	Optional server that holds the AI key, so installed copies need no key of their own. Supports an app token and request limits.
Supabase	Optional cloud storage for chat history, with each install seeing only its own chats.
electron-builder + NSIS	The one-click Windows installer and clean uninstaller.
Ollama PLANNED	Runs our own open models on our laptops.
Qwen3 8B / Llama 3.1 8B PLANNED	The local 8B text brain.
Small vision model PLANNED	A 3B vision model (such as Qwen2.5-VL 3B) for screenshots and images.
Whisper PLANNED	Offline speech-to-text.
Colab / Kaggle + Unsloth PLANNED	Free cloud GPUs and a fast library for fine-tuning.
VS Code extension API PLANNED	Native ghost text inside the editor, keeping the type-along teaching style.

7. Algorithms and techniques

The agent loop

Read the screen, decide one step, act, check, repeat, until the goal is reached or Lefty needs the user. One step at a time keeps every action checkable and stoppable.

Button matching (understanding "click save")

- The request is split into words, and filler words ("click", "please", "the", "button") are dropped.

- Each word is compared with each button name: an exact word scores 1, a prefix of 3+ letters scores 0.8 ("sett" matches "settings").
- Score = 70% **coverage** (how much of the request matched) + 30% **precision** (how much of the button name was used, so "Save" beats "Save a copy"), plus small bonuses for an exact name, an enabled button, and real button types.
- The best score above 0.5 wins. Long requests may miss one word.

Snapping (making the AI's pointing land on real things)

When the AI points at a spot on the screenshot, or picks a button whose name does not fit the step, Lefty moves the pointer to the real button whose name matches the step's words. Boxes Windows cannot see (such as VS Code's Extensions search box) are worked out from the buttons around them.

Click guard (safety)

Button names are checked against phrase rules with risk levels: **high** for things that cannot be undone ("permanently", "factory reset", "empty recycle bin"), **medium** for throwing away unsaved work ("don't save", "discard") or removing apps ("uninstall"). Safe words such as "undo", "cancel" and "restore" override, and minor targets such as "reset zoom" are ignored. Risky steps need the user's yes.

Model racing and fallback

Models are tried in order of speed and quality. The best is asked first; after 5 seconds a second is asked in parallel, and the first good answer wins. Errors such as "not found", "out of quota", "server error" and timeouts move on to the next model, with a short wait and an overall time budget when every model is busy.

Ghost-code tracking

The editor does not share its text, only the cursor's line and column. Lefty works out how much of each line has been typed from the cursor position minus the line's indentation, allowing for the editor's own auto-indent, and draws only the remaining code as faint text.

Voice

The microphone recording is turned into a small 16 kHz mono WAV file. Loudness (RMS) is measured to detect silence, so Lefty knows when you have stopped talking and sends the goal himself.

Focus mode

Each window that comes to the front is classified as work or not, from its app and title against the apps and topics you named. Anything else is closed the way its own close button would, so it can still ask to save.

Reliable start and uninstall

A single-instance lock stops two copies fighting over Ctrl+Space, and a newly installed version takes over from an old one. On uninstall, the installer leaves a note that Lefty checks every 2 seconds, so he closes himself even when running as administrator; his screen-reader helper exits on its own when its connection to Lefty closes.

8. What needs to be done (hackathon)

1. Make opening apps fast and reliable, and stop Lefty from saying "Done" when nothing happened.
2. When you ask for something specific, such as ghost code, that request wins over the mode switch.
3. A live status bubble that always shows what Lefty is doing right now.
4. A "+" button to upload a PDF or image, for example a question paper, with the answers shown in Lefty's main window.
5. Our own AI brain: an 8B open model on one of our laptops, connected to Lefty, with automatic fallback to Gemini.
6. Ghost code built properly inside VS Code as an extension, keeping the type-along teaching style.
7. Offline voice, so speaking to Lefty works without internet.
8. Demo preparation: a demo script, a recorded backup video, and a full rehearsal on our own phone hotspot.

Later, after the hackathon

- A PyCharm plugin with the same type-along ghost code.
- Terminal access, and a permission button: Ask every time, Auto, or Full access.
- Finding places (cafes and more) with location, price range, rating and his recommendation.
- Image generation for logos and website pictures.
- Fine-tuning our own model on Lefty's real, checked steps, with a new version each week.
- Teach by showing: do a task once, and Lefty can repeat it, as a shareable skill.
- A floating companion cursor over every app, clicks without moving your mouse, and a full redesign.

9. The demo story

1. Learn mode guides someone through a task in an app.
2. Switch to Do it, and Lefty finishes the task himself.
3. Type-along ghost code in VS Code.
4. Upload a question paper, and he answers every question.
5. Turn Wi-Fi off, and Lefty still works on our own brain.

Other assistants do it for you. Lefty teaches you to do it yourself, in any app, even offline.